

Lecture 8: Learning from images

Recommender Systems: Recap

■ Explicit feedback

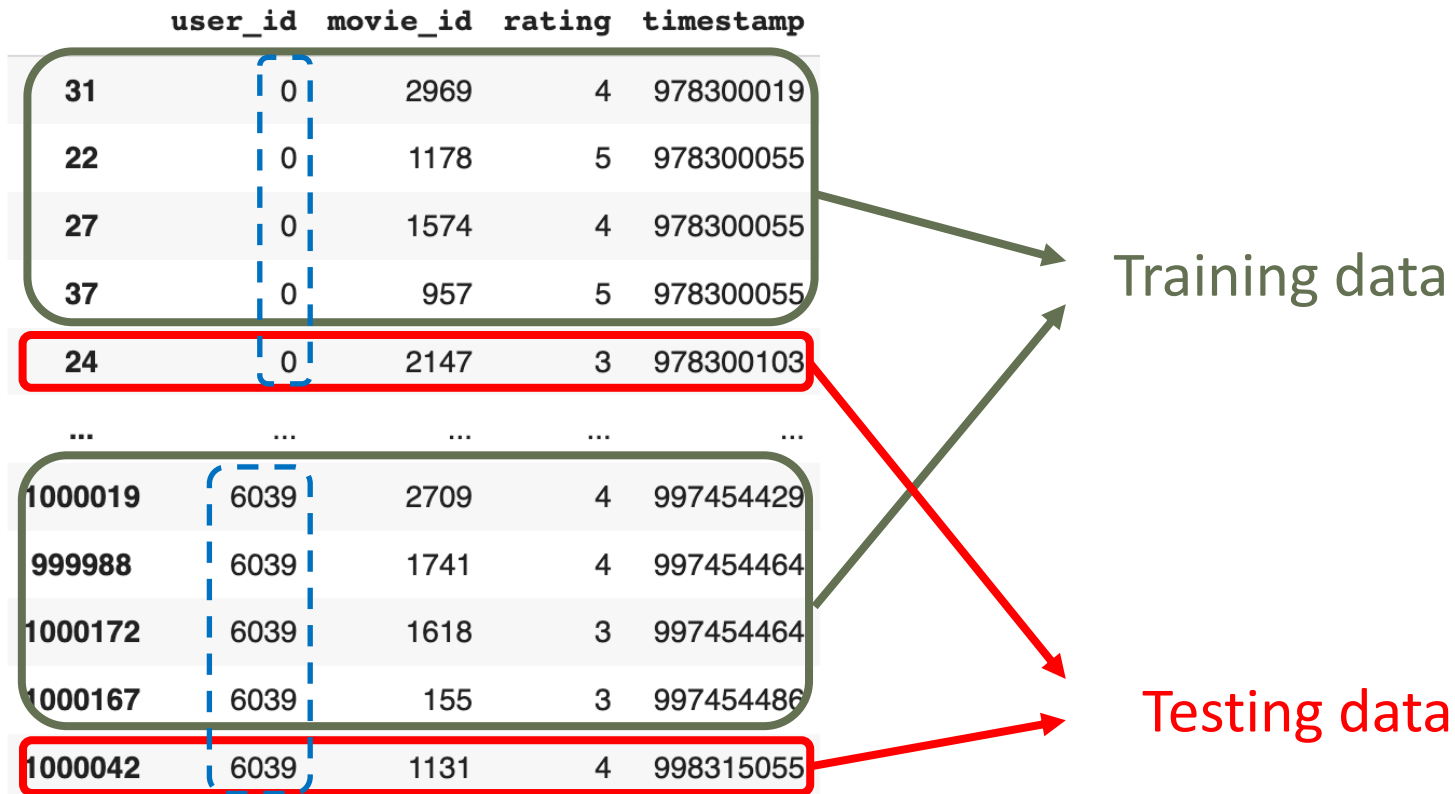
	<i>Batman Begins</i>	<i>Alice in Wonderland</i>	<i>Dumb and Dumber</i>	<i>Equilibrium</i>
User A	4	?	3	5
User B	?	5	4	?
User C	5	4	2	?
User D	2	4	?	3
User E	3	4	5	?

■ Implicit feedback

	<i>Item 1</i>	<i>Item 2</i>	<i>Item 3</i>	<i>Item 4</i>
User A	1	?	1	1
User B	?	1	1	?
User C	1	1	1	?
User D	1	1	?	1
User E	1	1	1	?

Recommender Systems: Recap

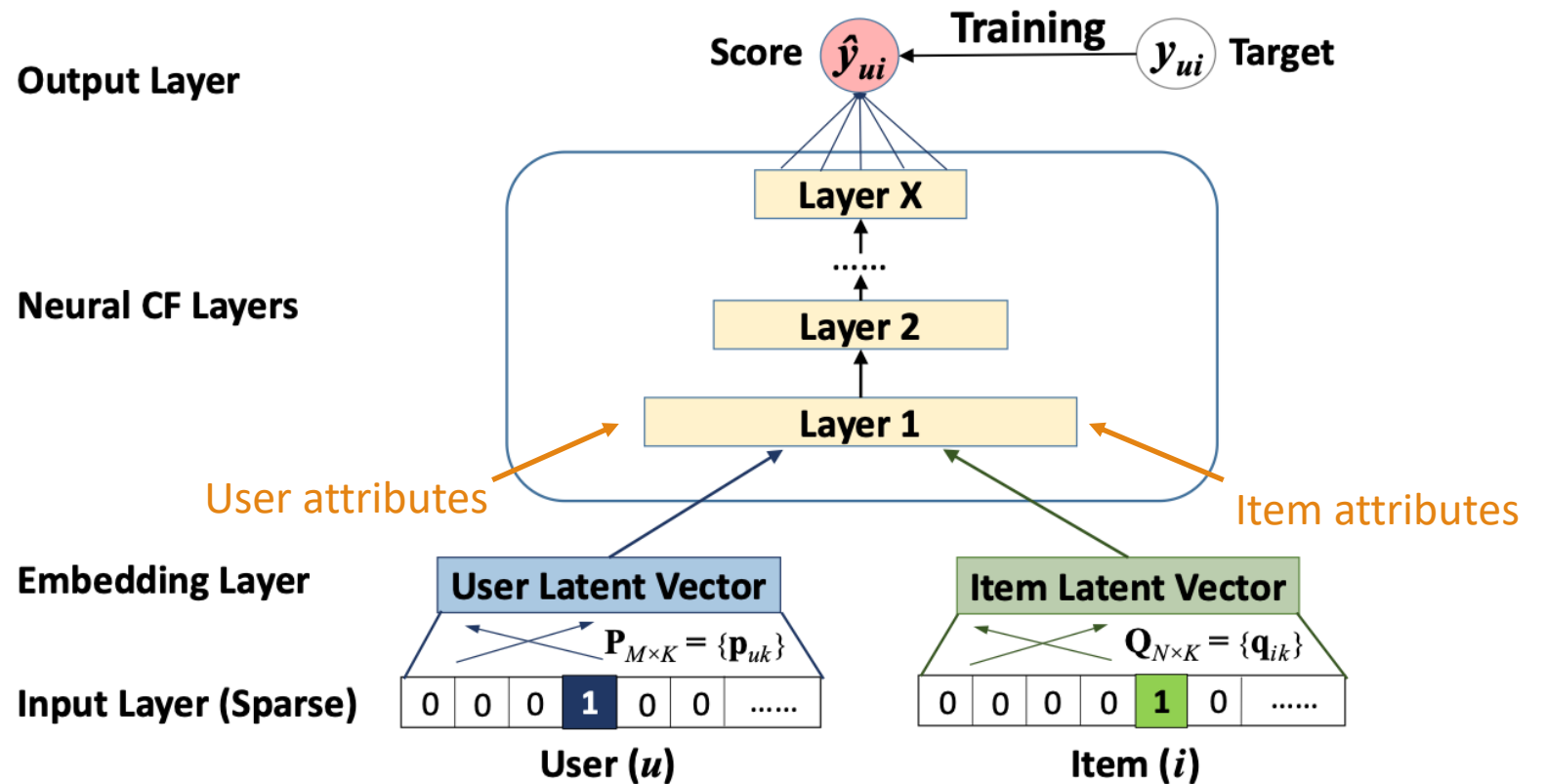
■ Partition the data



1000209 rows x 4 columns

Recommender Systems: Recap

- Extend Neural collaborative filtering with additional user and item attributes.



Overview

- Image data
- Represent image data with tensors
- A fully connected model for image classification

Image Data

- A picture is worth a thousand words
 - The introduction of convolutional neural networks revolutionized computer vision, and image-based systems have since acquired a whole new set of capabilities.
 - In order to participate in this revolution, we need to be able to load an image from common image formats and then transform the data into a tensor representation that has the various parts of the image arranged in the way PyTorch expects.

Business Applications Driven by Image Data

■ Image tagging

A company benefits from image tagging because of how quickly they organize images and how accessible their images become to users. When a user searches their website, they can find what they need based on simple keywords.



- In this image, the bigger words are stronger keywords and the smaller words are secondary.
- These were automatically sorted and ranked by an image tagging program.

Business Applications Driven by Image Data

■ Image classification

Photos for The Cheesecake Factory



The Cheesecake Factory

★★★★☆ 472 reviews

 Add photos

< **All (537)** Food (453) Inside (29) Drink (20) Menu (8) Outside (7)

> Search photos...



Business Applications Driven by Image Data

■ Image enhancement

- The researchers at UC Berkeley develop the automatic colorizer algorithm in 2016. It can colorize complex black and white photos accurately in seconds.



1. Image colorization attracts researchers in this field is Old Movies Colorizing.
2. Image or video improvement for effective advertising campaign

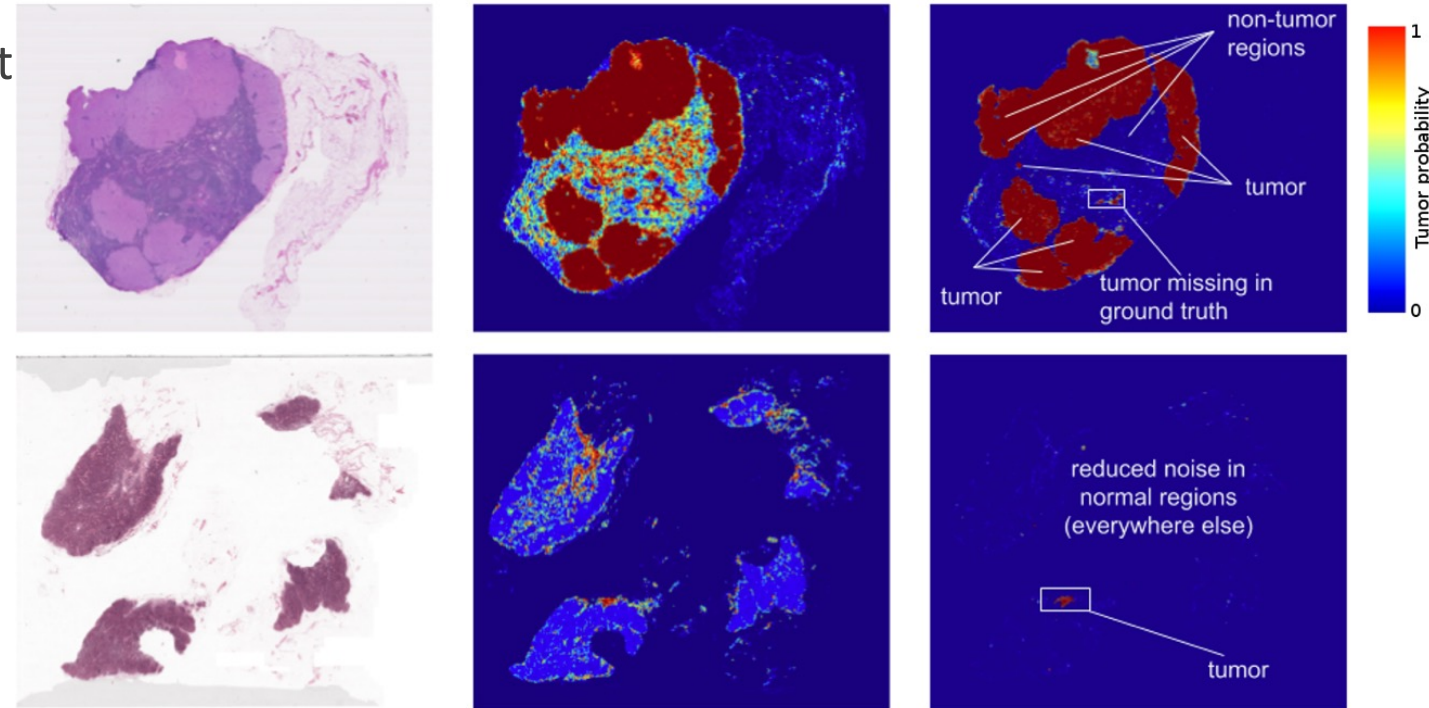
Business Applications Driven by Image Data

- Image generation



Business Applications Driven by Image Data

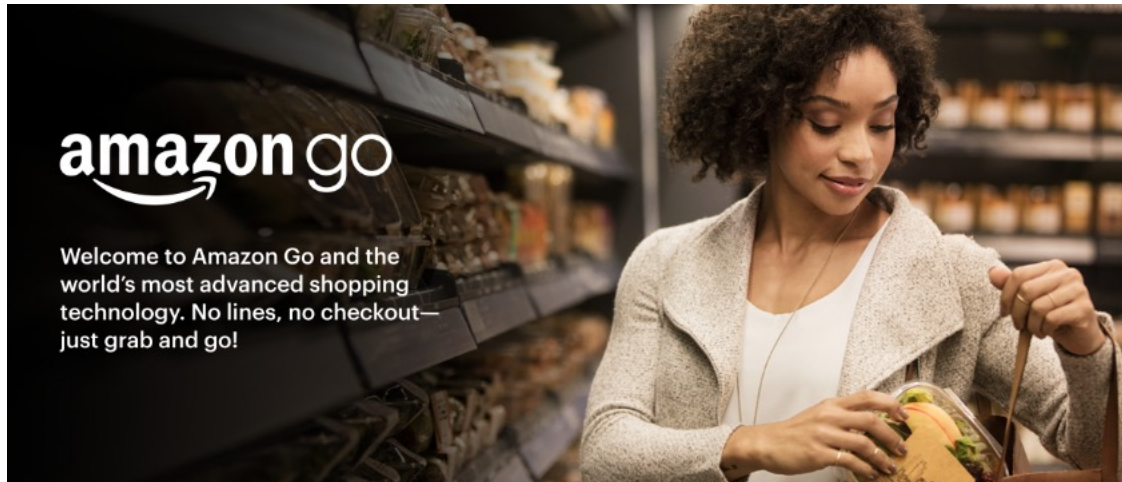
- Researchers at IBM estimate that medical images currently account for at least 90 percent of all medical data, making it the largest data source in the healthcare industry.
- **Inception v3** by Google is an example of AI-based image analysis that works with DNA and cancer data.
 - Developers used thousands of images of both healthy and cancerous tissues. **Inception v3** was able to advance on its own and learn the difference between the two types of tissues.
 - Inception v3 can also find mutations inside the DNA of cancer cells, helping doctors discover mutations and get ahead of the disease with appropriate treatment



The picture shows that deep learning-based image analysis can detect lymph node tumor

Business Applications Driven by Image Data

Amazon Go is an automated grocery stores that customers are able to purchase products without being checked out by a cashier or using a self-checkout station.

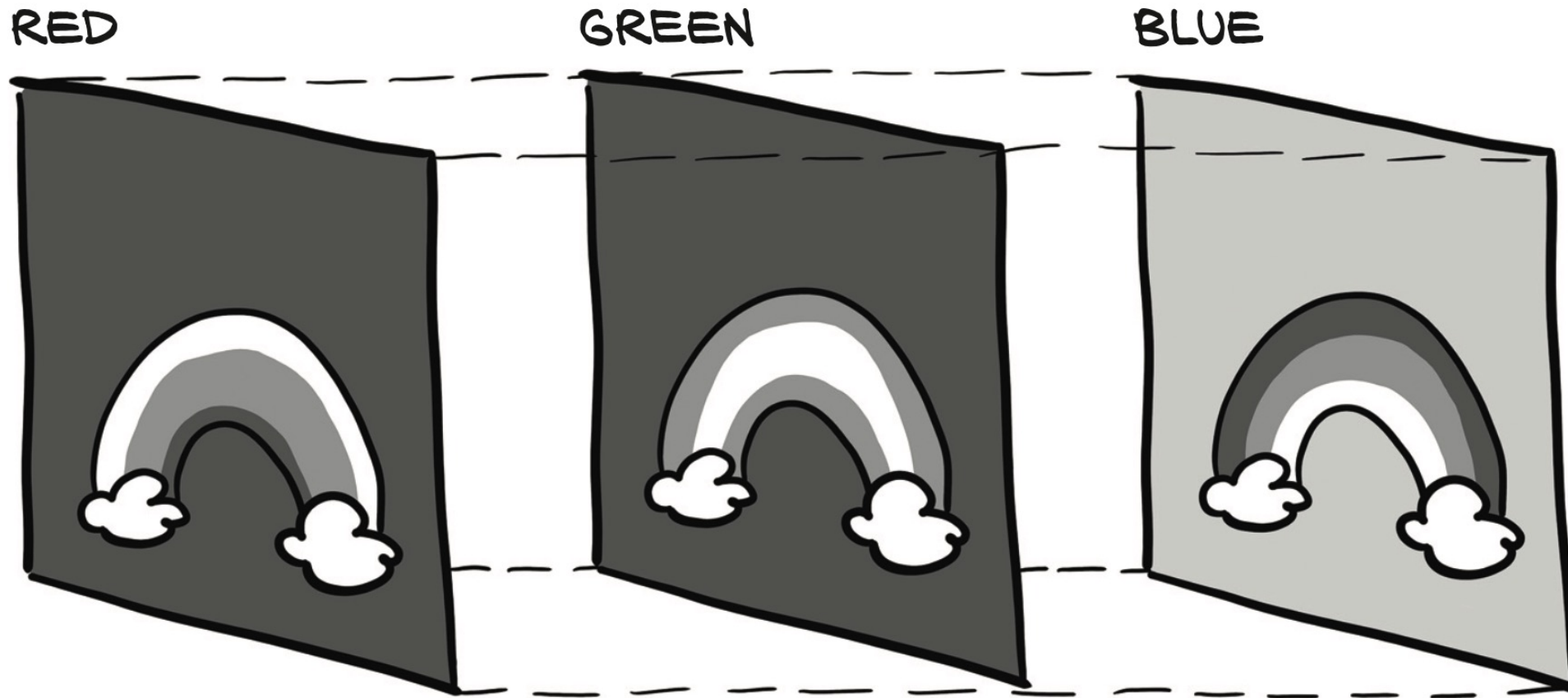


Go technology, a complicated array of cameras and software, figures out what shoppers have grabbed and automatically charges them when they exit.

Image Data

- An image is represented as a collection of scalars arranged in a regular grid with a height and a width (in pixels).
 - We might have a single scalar per grid point (the pixel), which would be represented as a grayscale image; or multiple scalars per grid point, which would typically represent different colors.
 - The most common is RGB, where a color is defined by three numbers representing the intensity of red, green, and blue.

Image Data

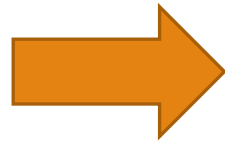
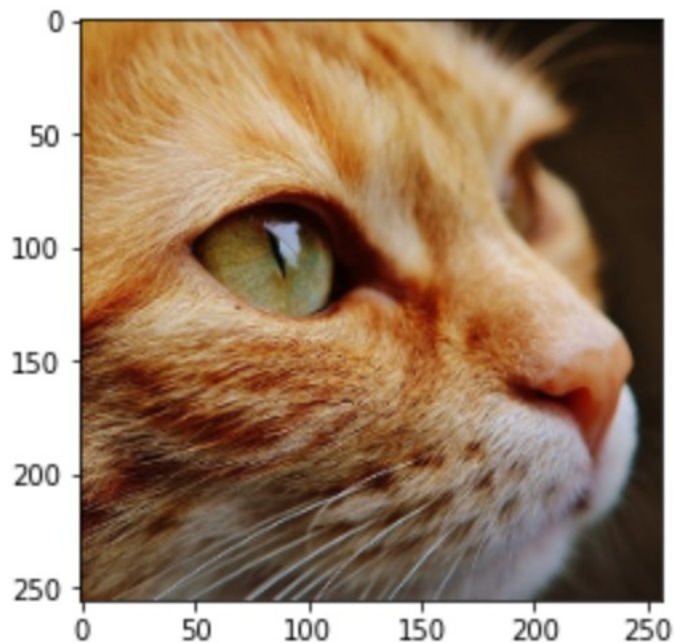


When you read an image into memory, the pixels usually have 8-bit integers between 0 and 255 for all three channels.

Each of the RGB channels captures a certain portion of the spectrum

Image Data

- PyTorch modules dealing with image data require tensors to be laid out as $C \times H \times W$: channels, height, and width, respectively.



```
tensor([[[[202, 193, ..., 13, 12],  
         [199, 192, ..., 14, 14],  
         ...,  
         [ 75,  68, ..., 36, 37],  
         [ 85, 103, ..., 37, 38]],  
        ...,  
        [[151, 139, ...,  9,  8],  
         [151, 140, ..., 11, 11],  
         ...,  
         [ 33,  26, ..., 26, 27],  
         [ 40,  58, ..., 27, 28]],  
        ...,  
        [[ 68,  53, ...,  6,  5],  
         [ 67,  54, ...,  6,  6],  
         ...,  
         [ 11,   2, ..., 17, 18],  
         [ 19,  37, ..., 18, 19]]], dtype=torch.uint8)
```

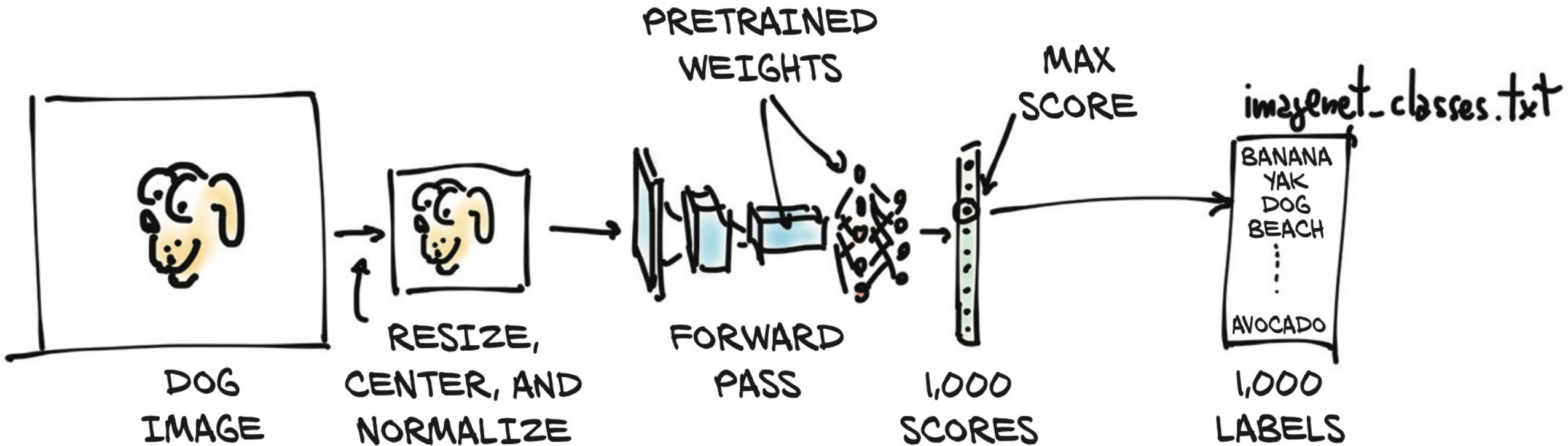
$C \times H \times W$: $3 \times 256 \times 256$

Each value is between 0 to 255

Steps to Build A Neural Network for Image Classification

- Step 1: Butterfly Data
- Step 2: Data Preprocessing
- Step 3: Build a Neural Network on Training Data
- Step 4: Evaluate Neural Network Performance

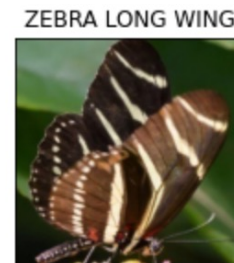
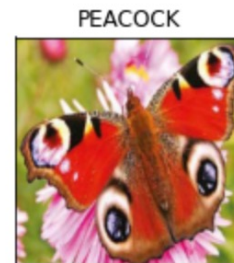
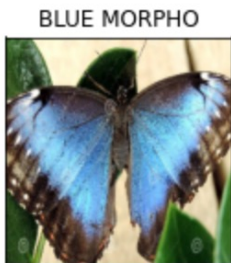
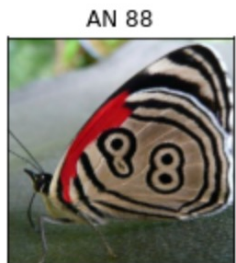
Steps to Build A Neural Network for Image Classification



- Image -> tensors -> image-classes

Step 1: Butterfly Image Data

- Butterfly Image Classification 75 species (<https://www.kaggle.com/datasets/gpiosenska/butterfly-images40-species>). All images are **224 X 224 X 3** in jpg format.
 - 10 species are selected



`torch.permute ()`: returns a view of the original tensor input with its dimensions permuted.

Step 2: Data Preprocessing

- Resize the input image to a given size

```
resized_imgs = [T.Resize(size=size)(orig_img) for size in (30, 50, 100, orig_img.size)]  
plot(resized_imgs)
```

Original image



Step 2: Data Preprocessing

- Convert an image to tensor
 - PyTorch modules dealing with image data require tensors to be laid out as $C \times H \times W$: channels, height, and width, respectively.

`torch.Size([3, 224, 224])`

red, green, and
blue channels

height

width

```
tensor([[[[0.3961, 0.3961, 0.4000, ..., 0.4353, 0.4275, 0.4314],
          [0.4039, 0.4039, 0.4000, ..., 0.4314, 0.4235, 0.4196],
          [0.4118, 0.4078, 0.4039, ..., 0.4431, 0.4353, 0.4275],
          ...,
          [0.0392, 0.0392, 0.0392, ..., 0.4706, 0.4941, 0.5020],
          [0.0392, 0.0392, 0.0392, ..., 0.4118, 0.4353, 0.4471],
          [0.0392, 0.0392, 0.0392, ..., 0.3608, 0.3804, 0.3882]],
        [[0.5294, 0.5294, 0.5333, ..., 0.5647, 0.5608, 0.5725],
          [0.5373, 0.5373, 0.5333, ..., 0.5647, 0.5569, 0.5608],
          [0.5451, 0.5412, 0.5373, ..., 0.5765, 0.5686, 0.5686],
          ...,
          [0.0431, 0.0431, 0.0431, ..., 0.5647, 0.5882, 0.5961],
          [0.0431, 0.0431, 0.0431, ..., 0.5216, 0.5451, 0.5569],
          [0.0431, 0.0431, 0.0431, ..., 0.4745, 0.4941, 0.5020]],
        [[0.3843, 0.3843, 0.3882, ..., 0.3412, 0.3255, 0.3333],
          [0.3922, 0.3922, 0.3882, ..., 0.3294, 0.3216, 0.3216],
          [0.4078, 0.4039, 0.4000, ..., 0.3412, 0.3333, 0.3255],
          ...,
          [0.0235, 0.0235, 0.0235, ..., 0.3608, 0.4000, 0.4078],
          [0.0235, 0.0235, 0.0235, ..., 0.2824, 0.3137, 0.3294],
          [0.0235, 0.0235, 0.0235, ..., 0.2118, 0.2353, 0.2510]]]])
```

Step 2: Data Preprocessing

- Normalize data

- Normalizing each channel so that it has the same distribution will ensure that channel information can be mixed and updated through gradient descent using the same learning rate
 - Calculate the mean and standard deviation from training data
 - Min-max normalization or Z-score standardization

```
transforms.Normalize(mean=[0.4621, 0.4528, 0.3400], std=[0.2884, 0.2767, 0.2862])
```

Step 2: Data Preprocessing

■ Normalize data

- Neural networks exhibit the best training performance when the input data ranges roughly from 0 to 1, or from -1 to 1 (this is an effect of how their building blocks are defined).
- So a typical thing we'll want to do is cast a tensor to floating-point and normalize the values of the pixels.

- Min-max normalization
- Z-score standardization

```
batch = batch.float()  
batch /= 255.0
```

```
n_channels = batch.shape[1]  
for c in range(n_channels):  
    mean = torch.mean(batch[:, c])  
    std = torch.std(batch[:, c])  
    batch[:, c] = (batch[:, c] - mean) / std
```

output has zero mean and
unit standard deviation
across each channel

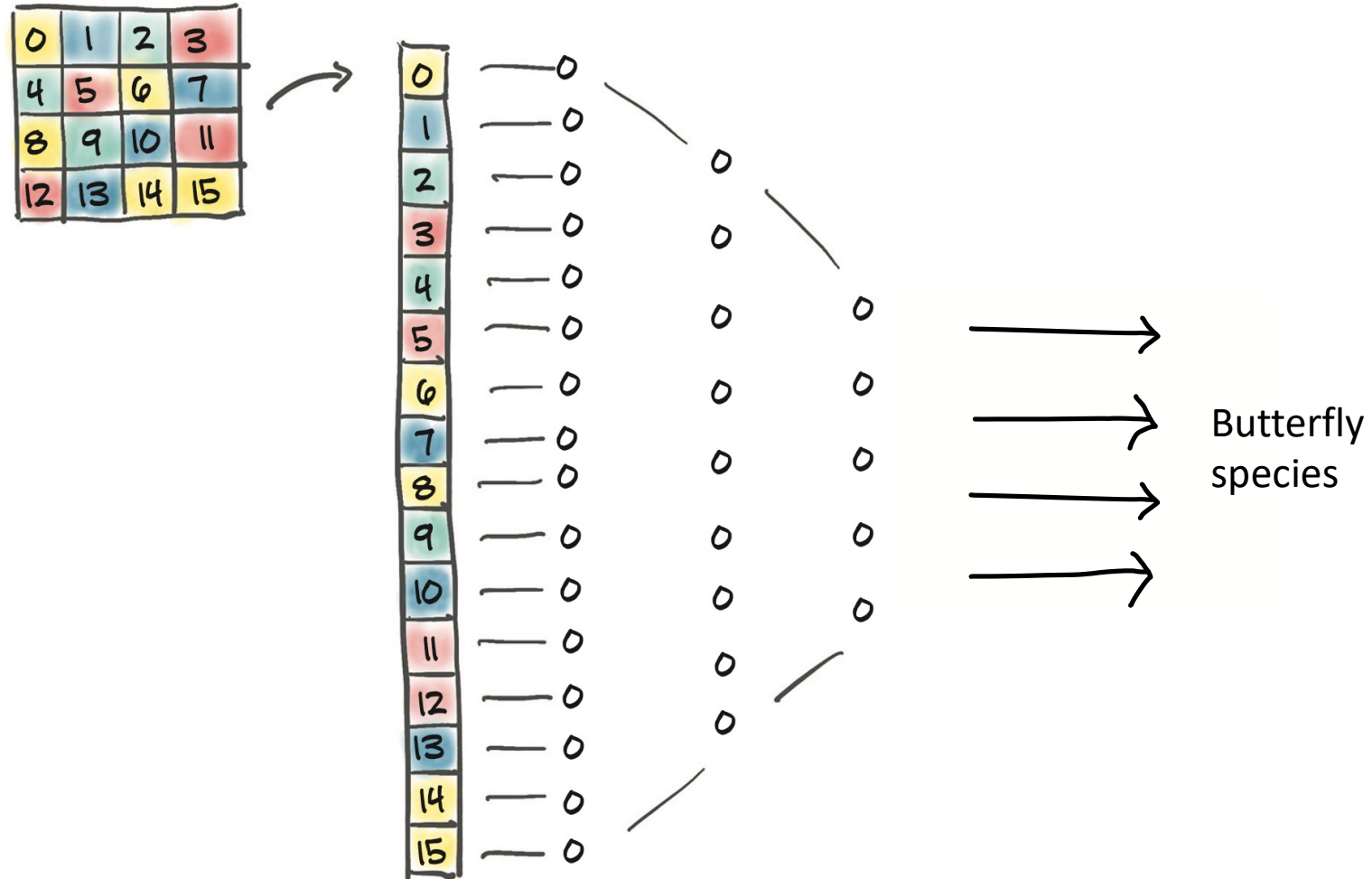
Step 2: Data Preprocessing

- More data preprocessing here:
https://pytorch.org/vision/main/auto_examples/plot_transforms.html

Step 3: Build a Neural Network on Training Data

- If we just take the image pixels and straighten them into a long 1D vector, we could consider those numbers as input features

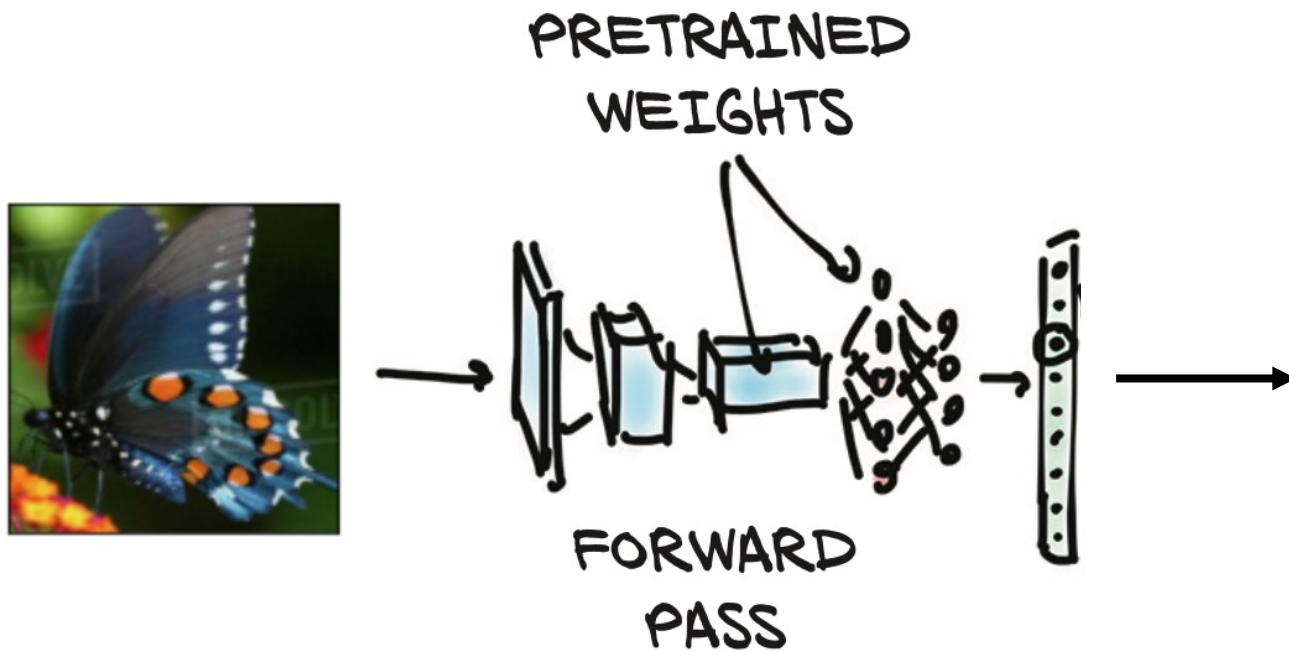
- $3 \times 224 \times 224$: that is, 150,528 input features per sample.



Step 3: Build a Neural Network on Training Data

- Multiclass classification
 - A classification task with more than two classes
- Loss function for classification (binary and multiclass classification)
 - Cross Entropy Loss
 - A measure from the field of information theory, building upon entropy
 - measures the performance of a classification model by calculating the difference between two probability distributions

Step 3: Build a Neural Network on Training Data



Score	Butterfly species
y_0	AN 88
y_1	BLUE MORPHO
y_2	COMMON WOOD-NYMPH
y_3	MONARCH
y_4	PEACOCK
y_5	PIPEVINE SWALLOW
y_6	ULYSES
y_7	VICEROY
y_8	YELLOW SWALLOW TAIL
y_9	ZEBRA LONG WING

Each element
between 0 and 1

$$\text{softmax}(y_0, y_1, \dots, y_9) = \left(\frac{e^{y_0}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}}, \frac{e^{y_1}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}}, \dots, \frac{e^{y_9}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}} \right)$$

Sum of element equals 1

Step 3: Build a Neural Network on Training Data

■ Cross Entropy Loss

- Measures the performance of a classification model by calculating the difference between two probability distributions

- Predicted probability distribution

$$\text{softmax}(y_0, y_1, \dots, y_9) = \left(\frac{e^{y_0}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}}, \frac{e^{y_1}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}}, \dots, \frac{e^{y_9}}{e^{y_0} + e^{y_1} + \dots + e^{y_9}} \right)$$

- Actual probability distribution

PEACOCK $\longrightarrow$ (0, 0, 0, 0, 1, 0, 0, 0, 0, 0)

$$\text{Cross Entropy} = -\frac{1}{N} \sum_j y_j * \log(\hat{y}_j) ; \quad y_j \text{ is the true label and } \hat{y}_j \text{ is the softmax probability}$$

Step 3: Build a Neural Network on Training Data

- Model training process
 - **1** Run the forward pass and obtain the output values from the last layer.
 - **2** Compute their softmax and obtain probabilities.
 - **3** Take the predicted probability corresponding to the correct class (the likelihood of the parameters). Note that we know what the correct class is because it's a supervised problem—it's our ground truth.
 - **4** Calculate the loss.
 - **5** Compute gradient and update parameters.

Step 3: Build a Neural Network on Training Data

- Training with minibatches

- The job of a data loader is to sample minibatches from a dataset

FOR N EPOCHS:

SPLIT DATASET IN MINIBATCHES

FOR EVERY MINIBATCH:

WITH EVERY SAMPLE IN MINIBATCH:

EVALUATE MODEL (FORWARD)

COMPUTE LOSS

ACCUMULATE GRADIENT OF LOSS (BACKWARD)

UPDATE MODEL WITH ACCUMULATED GRADIENT

Step 4: Evaluate Neural Network Performance

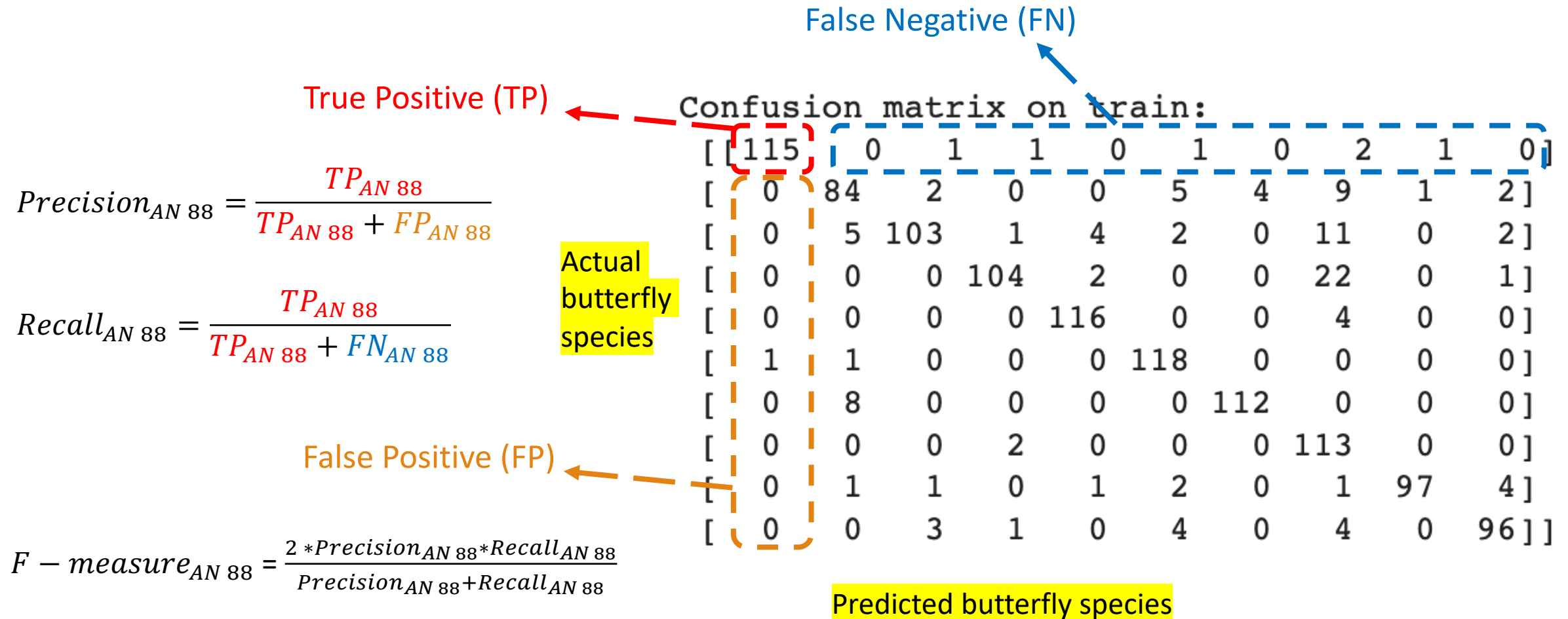
Classification report on train:

precision recall f1-score support

Confusion matrix on train:

[[115	0	1	1	0	1	0	2	1	0]	0	0.99	0.95	0.97	121	
[	0	84	2	0	0	5	4	9	1	2]	1	0.85	0.79	0.82	107
[	0	5	103	1	4	2	0	11	0	2]	2	0.94	0.80	0.87	128
[	0	0	0	104	2	0	0	22	0	1]	3	0.95	0.81	0.87	129
[	0	0	0	0	116	0	0	4	0	0]	4	0.94	0.97	0.95	120
[	1	1	0	0	0	118	0	0	0	0]	5	0.89	0.98	0.94	120
[	0	8	0	0	0	0	112	0	0	0]	6	0.97	0.93	0.95	120
[	0	0	0	2	0	0	0	113	0	0]	7	0.68	0.98	0.80	115
[	0	1	1	0	1	2	0	1	97	4]	8	0.98	0.91	0.94	107
[	0	0	3	1	0	4	0	4	0	96]]]	9	0.91	0.89	0.90	108
											ccuracy			0.90	1175
											macro avg			0.91	1175
											weighted avg			0.91	1175

Step 4: Evaluate Neural Network Performance



Step 4: Evaluate Neural Network Performance

■ Macro average:

$$Precision_{macro} = \frac{1}{10} (Precision_{AN\ 88} + Precision_{blue} + \dots + Precision_{zebra})$$

$$Recall_{macro} = \frac{1}{10} (Recall_{AN\ 88} + Recall_{blue} + \dots + Recall_{zebra})$$

$$F - measure_{macro} = \frac{1}{10} (F - measure_{AN\ 88} + F - measure_{blue} + \dots + F - measure_{zebra})$$

Balanced vs
imbalanced data

■ Weighted average:

$$Precision_{weighted} = \frac{(Precision_{AN\ 88} * N_{AN\ 88} + Precision_{blue} * N_{blue} + \dots + Precision_{zebra} * N_{zebra})}{Total\ number\ of\ samples}$$

$$Recall_{weighted} = \frac{(Recall_{AN\ 88} * N_{AN\ 88} + Recall_{blue} * N_{blue} + \dots + Recall_{zebra} * N_{zebra})}{Total\ number\ of\ samples}$$

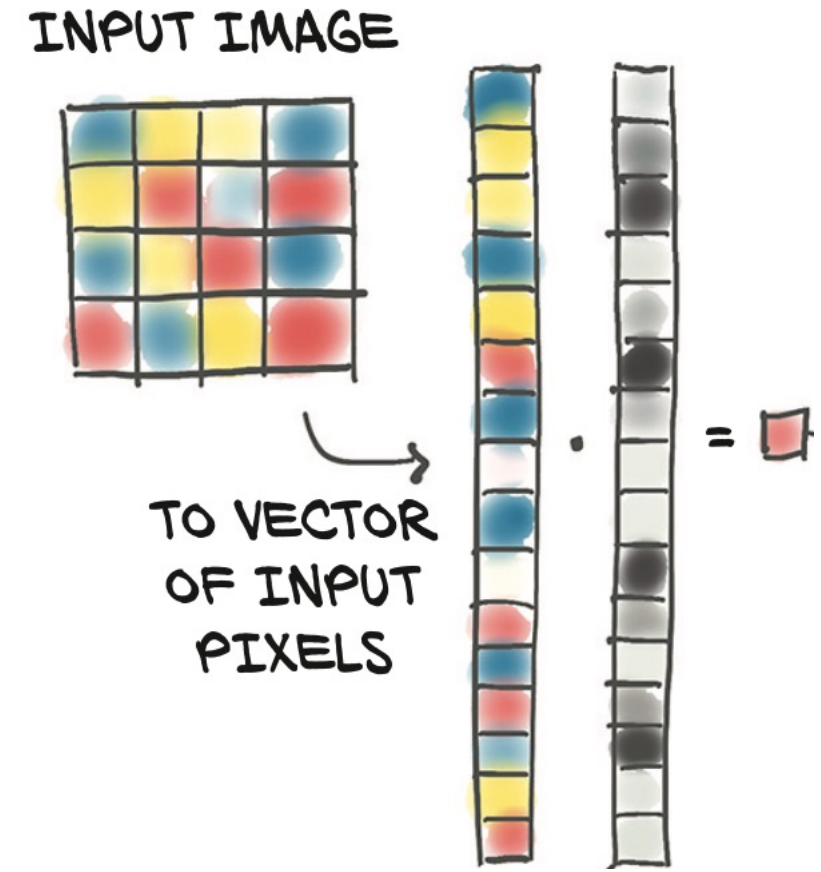
$$F - measure_{weighted} = \frac{(F - measure_{AN\ 88} * N_{AN\ 88} + F - measure_{blue} * N_{blue} + \dots + F - measure_{zebra} * N_{zebra})}{Total\ number\ of\ samples}$$

The limits of going fully connected

- Our neural network won't scale very well with the number of pixels.
 - What if we had a $1,024 \times 1,024$ RGB image? That's 3.1 million input values. Even abruptly going to 1,024 hidden features, we would have over 3 *billion* parameters. Using 32-bit floats, we're already at 12 GB of RAM, and we haven't even hit the second layer, much less computed and stored the gradients.

The limits of going fully connected

- On one hand, we are allowing for the combination of any pixel with every other pixel in the image being potentially relevant for our task. On the other hand, we aren't utilizing the relative position of neighboring or faraway pixels, since we are treating the image as one big vector of numbers.



The limits of going fully connected

- If we shift the same butterfly by one pixel or more, and the relationships between pixels will have to be relearned from scratch

